

CCR v2: A Survey and Roadmap for Persistent Agent Memory, Self-Evolving Strategies, and Sandboxed Execution in LLM-Based Coding Agents

CCR Research Team
github.com/ccr-project

March 10, 2026

Abstract

Large language model (LLM) based coding agents face three fundamental challenges: (1) maintaining coherent memory across sessions, (2) evolving task strategies from experience, and (3) safely executing code for iterative reasoning. We present a comprehensive survey of 50+ recent papers (2025–2026), frameworks, and production systems across these three domains, conducted to inform the next version of CCR (Claude Context Reducer)—an MCP server that unifies Git-style versioned memory (GCC), self-evolving playbooks (ACE), and sandboxed REPL execution (RLM) for Claude Code. Our analysis identifies 14 high-impact improvements organized into three priority tiers, backed by evidence from ICLR 2026, ACL 2025, NeurIPS 2025, and industry systems. We find that (a) graph-based memory with admission control outperforms flat stores by up to 45.5% on reasoning benchmarks, (b) hierarchical skill libraries with structured failure distillation achieve 10–20× compression over raw trajectories, (c) kernel-level sandboxing (Landlock/Seatbelt) provides zero-overhead isolation superior to application-level blocklists, and (d) CCR’s unified architecture remains unique in the ecosystem—no competing system combines versioned memory, self-evolving strategies, and sandboxed execution in a single zero-LLM-call MCP server. We propose a concrete roadmap for CCR v2 that integrates these findings while preserving the system’s zero-infrastructure, zero-API-key design philosophy.

1 Introduction

LLM-based coding agents—systems such as Claude Code [30], Cursor, Devin [36], and OpenHands [34]—have rapidly advanced from single-turn code completion to autonomous multi-step software engineering. However, three persistent limitations constrain their effectiveness in long-horizon, multi-session workflows:

Memory amnesia. Most coding agents begin each session with no knowledge of prior work. Claude Code’s native Auto Memory (MEMORY.md) and Cursor’s community-built Memory Banks offer flat markdown persistence, but lack version control, branching for experimental isolation, or structured retrieval beyond keyword search.

Strategy stasis. Agents repeatedly encounter the same failure modes across sessions without learning from them. While reinforcement learning from human feedback (RLHF) adapts model weights, it cannot capture project-specific heuristics, coding conventions, or debugging insights that vary across codebases.

Unsafe execution. Code execution for iterative reasoning—validated by the RLM paradigm [3]—requires sandboxing that balances security with zero-overhead performance. Current approaches range from heavy-weight microVMs (E2B, Firecracker) to fragile application-level blocklists.

CCR (Claude Context Reducer) addresses all three challenges through a unified MCP server exposing 17 tools built on three research papers: GCC (Git Context Controller) [1] for versioned memory, ACE (Agentic Context Engineering) [2] for self-evolving playbooks, and RLM (Recursive Language Models) [3] for sandboxed REPL execution. CCR requires zero LLM calls, zero infrastructure, and zero API keys—operating entirely through Claude Code’s native tool-calling interface on a Claude Max subscription.

This report surveys the state of the art across all three domains as of March 2026, identifies gaps in CCR’s current implementation relative to recent advances, and proposes a prioritized roadmap for CCR v2. Our contributions are:

1. A structured survey of 50+ papers, frameworks, and production systems across agent memory, context engineering, and sandboxed execution (Sections 2–4).
2. A competitive analysis of the agent memory ecosystem including Claude Code native features, IDE agents, and the MCP marketplace (Section 5).
3. A three-tier improvement roadmap with impact/effort ratings backed by empirical evidence (Section 6).

2 Related Work: Agent Memory Systems

The study of persistent memory for LLM agents has matured from scattered proposals to a recognized research area, with a dedicated ICLR 2026 workshop (MemAgents) [20] and comprehensive surveys [18, 19].

2.1 Taxonomies and Theoretical Foundations

The most comprehensive taxonomy is proposed by [18], who organize agent memory along three dimensions: **Forms** (token-level, parametric, latent), **Functions** (factual, experiential, working), and **Dynamics** (formation, evolution, retrieval). Experiential memory is further divided into case-based (specific episodes), strategy-based (general approaches), and skill-based (reusable procedures). CCR’s GCC commits correspond to case-based memory, while ACE playbook bullets correspond to strategy-based memory. The survey identifies skill-based memory as underexplored—CCR’s CODE SNIPPETS section partially addresses this but lacks structured preconditions, expected outcomes, and failure modes.

The Continuum Memory Architecture (CMA) [16] formalizes five architectural requirements for cross-session state: **persistent storage**, **selective retention**, **associative routing**, **temporal chaining**, and **consolidation into higher-order abstractions**. Evaluating CCR against these requirements reveals two gaps: no admission control (selective retention) and limited graph-based retrieval (associative routing).

2.2 Graph-Based Memory Architectures

MAGMA [7] decouples memory into four orthogonal graphs—semantic, temporal, causal, and entity—with policy-guided traversal at retrieval time. A dual-stream write mechanism separates fast ingestion from asynchronous consolidation. Results show up to 45.5% higher reasoning accuracy, 95% token reduction, and 40% faster query latency versus flat retrieval. This directly informs a potential upgrade to GCC’s commit structure: maintaining parallel graph indexes over commits would enable queries like “what caused this bug?” (causal graph) or “what have we done to this file?” (entity graph).

SYNAPSE [8] applies spreading activation from the ACT-R cognitive architecture [43] to agent memory. Relevance emerges dynamically through activation propagation rather than pre-computed static links, with lateral inhibition suppressing weakly related memories and temporal decay aging out stale information. This solves the “Contextual Tunneling” problem where vector similarity retrieval fixates on surface-level matches, missing deeper causal connections.

A-MEM [9] applies the Zettelkasten method to agent memory, where each note carries structured attributes (keywords, tags, contextual descriptions) and explicit bidirectional links. Critically, new experiences *retroactively refine* existing notes’ attributes, making memory a living graph rather than an append-only log. GCC commits are currently write-once; A-MEM suggests that new commits should trigger metadata updates to related earlier commits.

Graphiti/Zep [10] introduces a bi-temporal data model tracking both event occurrence time and ingestion time, enabling point-in-time queries (“what did the agent know at time T?”). With P95 latency of 300ms and zero LLM calls at retrieval time, this approach aligns with CCR’s zero-LLM-call philosophy.

PlugMem [11] (Microsoft Research, ICML 2026 under review) structures episodic memories into a compact knowledge-centric graph where propositional and prescriptive knowledge are the units of access. Information-theoretic analysis confirms highest information density across task types. This suggests extracting atomic knowledge from GCC commits into a separate lightweight index.

2.3 Memory Consolidation and Forgetting

A-MAC [12] treats memory admission as a structured decision problem with five scoring factors: future utility, factual confidence, semantic novelty, temporal recency, and content type prior. Ablation reveals content type prior as the single most influential factor. F1 of 0.583 on LoCoMo with 31% latency reduction. This directly motivates adding a novelty gate to `gcc_commit`.

TiMem [13] organizes memory in a Temporal Memory Tree (TMT) with 5 hierarchical levels, from raw conversation to progressively abstracted representations. Semantic-guided consolidation promotes important details upward. Achieves 52.2% reduction in recalled memory length while maintaining 75.3% accuracy on LoCoMo. This informs an upgrade to GCC’s single rolling summary.

HiMem [14] introduces *conflict-aware memory reconsolidation*: when retrieval surfaces old memories that contradict current state, the system detects and revises stored knowledge. This addresses a real problem in long-running projects where stale information misleads future sessions.

Temporal decay mechanisms inspired by cognitive architectures [44] provide principled forgetting without deletion. Under exponential decay ($\text{score}_{\text{eff}} = \text{score}_{\text{raw}} \cdot 0.95^{\text{days}}$), a strategy unused for 30 days retains $\sim 21\%$ of its original weight, naturally deprioritizing stale entries.

2.4 Benchmarks

The agent memory evaluation landscape has consolidated around several benchmarks: **MemoryAgent-Bench** [21] (ICLR 2026) evaluates retrieval accuracy, test-time learning, long-range understanding, and selective forgetting; **MemBench** [22] (ACL 2025 Findings) evaluates effectiveness, efficiency, and capacity across factual and reflective memory; **LoCoMo** [23] (Snap Research) has become the de facto standard for long-term conversational memory, used by MAGMA, TiMem, SYNAPSE, A-MAC, and Zep; and **AMA-Bench** [24] focuses specifically on long-horizon agentic memory.

3 Context Engineering and Self-Evolving Strategies

3.1 Beyond ACE: Meta Context Engineering

The most significant advance beyond ACE is **MCE** (Meta Context Engineering via Agentic Skill Evolution) [4]. While ACE evolves playbook *content* (add, update, merge, remove bullets within a fixed schema), MCE evolves the context engineering *process itself*—the instructions and code that govern how context is constructed.

MCE operates at two levels: a **meta-level agent** that maintains a library of CE “skills” (executable instructions + code) and performs *agentic crossover*—deliberative search over the history of skills, their executions, and evaluations—and a **base-level agent** that executes evolved skills to construct context as programmatic artifacts. Results show 5.6–53.8% relative improvement over state-of-the-art agentic CE methods (mean 16.9%), with superior transferability across task families.

The implication for CCR is profound: the playbook format (sections with slugged bullets and counters) could itself be treated as an evolvable artifact, with a meta-level process periodically evaluating whether the current structure serves well.

3.2 Hierarchical Skill Libraries

SkillRL [5] introduces a hierarchical SkillBank with three key mechanisms:

1. **Distillation:** Successful trajectories are compressed into strategic patterns (10–20× compression). Failed trajectories are distilled into structured *failure lessons* with four components: failure point, flawed reasoning, correct counterfactual, and general prevention principle.
2. **Hierarchy:** General skills (universal strategies) versus task-specific skills (procedural guides with nuanced preconditions).
3. **Recursive evolution:** Validation checkpoints detect when accumulated failures expose knowledge gaps, triggering targeted skill refinement.

This directly informs CCR’s ACE system. Currently, a harmful increment (`harmful=3`) conveys no information about *why* a strategy failed or *what should have been done instead*. SkillRL’s structured failure lessons would make harmful entries actionable.

EXIF [6] introduces *exploration-first* skill discovery using dual agents: an explorer that proactively generates skill datasets by interacting with environments, and a learner that trains on discovered skills. When both roles are filled by the same model, the system becomes self-evolving. This contrasts with CCR’s current reactive approach where insights are accumulated only when they arise during normal work.

3.3 Context Compression

ACON [25] (ICLR 2025) learns compression guidelines from paired trajectory analysis—comparing where full context succeeds versus compressed context fails. An LLM analyzes failure causes and iteratively updates compression guidelines in natural language. Results: 26–54% peak token reduction, with distilled compressors retaining 95% accuracy.

Anthropic’s own context engineering principles [30] emphasize that “every token competes for attention” and recommend the smallest possible set of high-signal tokens. The multi-agent summarization pattern—subagents explore extensively but return condensed 1–2K token summaries—aligns with CCR’s RLM design.

CER (Contextual Experience Replay) [26] (ACL 2025) demonstrates training-free self-improvement through a dynamic memory buffer of synthesized experiences, achieving 51% relative improvement over GPT-4o. CER’s training-free approach validates CCR’s zero-LLM-call philosophy while suggesting that `gcc_commit` could include a `patterns_learned` field capturing transferable decision patterns.

3.4 Retrieval-Augmented Generation for Code

A-RAG [27] presents hierarchical retrieval with three tools (keyword search, semantic search, chunk read) where models dynamically choose retrieval strategies. This outperforms fixed pipelines because different queries benefit from different strategies. CCR’s `index_search` currently supports only keyword/symbol search; adding semantic search would enable conceptually related code discovery.

4 Sandboxed Execution and REPL-Based Reasoning

4.1 The RLM Paradigm and Its Validation

The RLM (Recursive Language Models) paradigm [3] treats long prompts as an external environment, giving the LLM a Python REPL with context loaded as a string variable. Key results include handling inputs up to 2 orders of magnitude beyond context windows, 29% gain on BrowseComp-Plus with GPT-5, and 58% F1 on OOLONG-Pairs.

Critically, the *REPL-only ablation* (no recursive sub-model calls) still improves performance on several tasks, validating CCR’s architecture where Claude Code—the model itself—drives the iteration loop. Post-paper advances include **RLM-Qwen3-8B** [32] (the first natively recursive model, +28.3% over vanilla Qwen3-8B) and Prime Intellect’s **RLMEnv** [33] (confirming consistent gains across four benchmarks).

4.2 The Sandbox Security Hierarchy

The sandboxing landscape has stratified into a clear hierarchy, which we present in Table 1.

Table 1: Sandbox isolation tiers for LLM agent code execution, ordered from strongest to weakest isolation. CCR currently operates at the weakest tier (Tier 6).

Tier	Technology	Boot	Overhead	Representative
1	Firecracker microVM	125ms	3–5 MB	E2B, AWS Lambda
2	Kata Containers	200ms	~10 MB	Northflank
3	gVisor	50ms	~15 MB	Modal, GKE
4	Wasm (Wasmtime)	<13ms	<5 MB	Wassette [38]
5	Kernel filters	<1ms	Zero	nono [39]
6	Process blocklists	<1ms	Zero	CCR (current)

Nono [39] uses Landlock (Linux) and Seatbelt (macOS) for kernel-level restriction with deny-by-default

semantics. It auto-blocks SSH keys, cloud credentials, and shell history, provides atomic rollback with cryptographic audit chains (Sigstore attestation, in-toto/SLSA compliance), and critically, “once restrictions are applied, there is no API to escape them—not even for nono itself.” This is directly applicable to CCR on macOS (M4 Pro) with zero performance overhead.

Wassette [38] (Microsoft) runs WebAssembly Components via MCP with a deny-by-default capability system. No filesystem, network, or environment access unless explicitly granted. Sub-13ms startup. The capability model is formally similar to browser sandboxes, providing strong isolation without virtualization overhead.

CaMeL [37] (Google DeepMind) addresses a complementary threat—prompt injection via data flow control. A Privileged LLM orchestrates tasks while a Quarantined LLM (stripped of tool-calling) processes untrusted data. Every value carries provenance tags, and policies prevent tainted data from flowing into irreversible actions. CaMeL solves 77% of tasks with provable security versus 84% undefended (only 7% capability cost).

The **OWASP Top 10 for Agentic Applications** (2026) [40] identifies Agent Goal Hijacking (ASI01) and Tool Misuse (ASI02) as the top risks, emphasizing the Least Agency principle: autonomy should be earned, not default.

4.3 MCP Server Patterns

The MCP specification has evolved significantly through 2025–2026, adding streamable HTTP transport, OAuth 2.1 authorization, **tool annotations** (readOnlyHint, destructiveHint, idempotentHint), **output schemas**, and **elicitation** (servers requesting structured input mid-execution).

The **Cloudflare Code Mode** [41] pattern is particularly relevant: instead of exposing every API operation as a separate MCP tool (consuming >1M tokens for large APIs), expose just `search()` and `execute()`. The agent discovers capabilities via search and writes code against a typed SDK. Result: 81% fewer tokens versus direct tool calling. This aligns with CCR’s REPL philosophy—give the agent a programmable interface rather than enumerated tools.

Anthropic’s **Programmatic Tool Calling** [31] achieves 98.7% token reduction (150K to 2K tokens) by shifting from sequential tool calling to code-based orchestration. Their **Tool Search Tool** enables searching thousands of tools without loading all definitions upfront (85% token reduction).

5 Competitive Landscape Analysis

We analyze the competitive positioning of CCR against six categories of systems. Table 2 summarizes capabilities.

Claude Code Native Memory. Ships with CLAUDE.md (user-written instructions) and Auto Memory (agent-written MEMORY.md). Strengths: zero setup, auditable, survives compaction. Weaknesses: no version history, no branching, no structured metadata, no self-evolving strategies, 200-line auto-load limit.

IDE Agents (Cursor, Windsurf, Cline, Roo Code). Windsurf offers the best native memory (Memories & Rules), while Roo Code Memory Bank provides structured categories (decisionLog, systemPatterns). None offer version control over memory or self-evolving strategies. All rely on MCP marketplace for extensibility.

Autonomous Agents (Devin, OpenHands, SWE-Agent, Aider). OpenHands’ *context condensation* [35]—

Table 2: Capability comparison across agent memory and context systems (March 2026). CCR is the only system combining all six capabilities.

Capability	Claude Native	IDE Agents	Devin/OH	Lang-Graph	MCP Servers	CCR
Cross-session memory	Flat	Via MCP	Events	Store	Various	Git
Version control	–	–	–	–	–	✓
Self-evolving strategies	–	–	–	–	–	✓
Sandboxed REPL	–	–	Env	–	–	✓
Zero LLM calls	✓	N/A	–	–	Mostly –	✓
Zero infrastructure	✓	Varies	–	–	Varies	✓

summarizing older events while preserving recent ones—achieves $\sim 50\%$ cost reduction without accuracy loss and linear (vs. quadratic) scaling. Devin Wiki auto-indexes repositories. Neither system has self-evolving strategies or versioned memory.

Agent Frameworks (LangGraph, CrewAI, AutoGen, Semantic Kernel). LangGraph offers the most mature memory with namespaced document stores, TTL support, and multiple backends (Postgres, Redis, MongoDB). CrewAI provides structured memory types (entity, crew). All require infrastructure and are designed for custom applications, not coding agents.

MCP Memory Servers. The marketplace has reached 8,500+ servers. Notable entries: `claude-mem` (ChromaDB + AI compression, requires API keys), `memsearch` (Milvus + hybrid search, requires embedding API), `engram` (SQLite + FTS5, agent-agnostic), `mcp-memory-service` (knowledge graphs, `sqlite-vec`). None combine versioned memory, self-evolving strategies, and sandboxed execution.

Convergence signal. Letta/MemGPT’s February 2026 update introduced “Context Repositories”—git-based versioning for memory—independently validating GCC’s core design choice. GCC v2 [1] (revised March 2026) reports 13%+ improvement on SWE-Bench Verified with 80%+ success rate, outperforming 26 open and commercial systems.

CCR’s unique moats: (1) Only system implementing all three papers (GCC + ACE + RLM) as a unified MCP server; (2) Zero LLM calls—works with Claude Max, no API keys; (3) Git-style memory with branch/merge is fundamentally more powerful than flat storage; (4) Self-evolving playbooks with helpful/harmful counters have zero competitors; (5) 17 tools in one MCP server versus competitors’ 3–5; (6) Pure file-based, zero infrastructure.

6 CCR v2 Roadmap

Based on the survey, we propose 14 improvements organized into three priority tiers by impact-to-effort ratio.

6.1 Tier 1: High Impact, Low–Medium Effort

1. Structured Failure Lessons (ACE). *Source: SkillRL [5].* Replace bare `harmful=N` counter increments with structured records containing four fields: failure point, flawed reasoning, correct counterfactual, and general prevention principle. This makes harmful entries actionable rather than merely flagged. Expected impact: 10–20 $\times$ information density per harmful entry.

2. Memory Admission Control (GCC). *Source: A-MAC* [12]. Before writing a new commit, compute semantic overlap with the last k commits (file intersection + keyword similarity). If overlap exceeds a threshold τ (e.g., 0.8), auto-merge into the previous commit. This prevents the “50 commits that all say continued working on X” pattern. Expected impact: 30–50% reduction in redundant commits.

3. Temporal Decay for Playbook Counters (ACE). *Source: SYNAPSE* [8], *ACT-R* [43]. Apply exponential decay to helpful/harmful counters:

$$\text{score}_{\text{eff}} = \text{score}_{\text{raw}} \cdot \lambda^d \quad (1)$$

where $\lambda = 0.95$ and d is days since last update. A strategy unused for 30 days retains $\lambda^{30} \approx 0.21$ of its score. This implements principled forgetting without deletion.

4. Two-Tier Playbook (ACE). *Source: SkillRL* [5], *Memory Survey* [18]. Split the playbook into:

- **Global:** `~/.ccr/global_playbook.txt`—universal heuristics that transfer across projects (e.g., “validate assumptions at each step”).
- **Project:** `.ccr/playbook.txt`—project-specific strategies (e.g., “ChatMessage is a struct, not an enum in ironclaw”).

Both are injected via the session start hook. Global strategies persist when switching projects, enabling cross-project transfer learning.

5. Kernel Sandboxing (RLM). *Source: nono* [39], *OWASP* [40]. Add macOS Seatbelt enforcement around the Python REPL subprocess with deny-by-default filesystem and network access. Auto-block `~/.ssh`, `~/.aws`, `~/.config/gcloud`, and shell history. Zero performance overhead; single wrapper function around existing subprocess launch.

6. MCP Tool Annotations. *Source: MCP Spec 2025-11-25* [42]. Add `readOnlyHint`, `destructiveHint`, and `idempotentHint` metadata to all 17 tools. Zero code change to logic—pure metadata. Enables Claude Code to make smarter safety decisions about tool execution ordering and confirmation requirements.

6.2 Tier 2: High Impact, Medium Effort

7. Multi-Level Hierarchical Summaries (GCC). *Source: TiMem* [13]. Upgrade the single rolling summary to three tiers: session summaries (every 5–10 commits), phase summaries (on branch merges or milestones), and project overview (monthly). `gcc_context(level=1)` returns phase summaries; higher levels add progressively more detail. Expected token reduction: ~52% for routine context loads.

8. Retroactive Commit Linking (GCC). *Source: A-MEM* [9], *MAGMA* [7]. When `gcc_commit` creates a new entry, scan recent commits for semantic overlap (shared files, referenced concepts) and store cross-links as metadata. Link types: entity (same files), causal (“why” references a prior problem), and supersession (replaces an earlier approach). Enables graph-based traversal in `gcc_context`.

9. Evolving Compression Guidelines (GCC/ACE). *Source: ACON* [25]. Maintain `.ccr/compression_guidelines` capturing what information must survive compaction. When information loss causes problems (agent cannot find previously committed data), the guidelines are updated. Over time, compaction learns what to preserve for each project.

10. Patterns Field in Commits (GCC). *Source: CER* [26]. Add `patterns_learned` to `gcc_commit` alongside existing fields. Captures transferable decision patterns (e.g., “tokio async tests require special Mutex handling”) that are candidates for automatic promotion to ACE playbook bullets.

6.3 Tier 3: Transformative, Higher Effort

11. Meta-Level Playbook Evolution (ACE). *Source: MCE* [4]. Treat the playbook format itself—section structure, counter thresholds, delta operations—as an evolvable artifact. A meta-level process periodically evaluates whether the current structure serves well and proposes structural changes. Expected impact: 5.6–53.8% improvement based on MCE results.

12. Wasm Sandbox Migration (RLM). *Source: Wassette* [38], *agent-sandbox*. Replace the Python REPL’s process-level blocklist with a WebAssembly capability-based sandbox. Sub-13ms startup, deny-by-default for filesystem/network/env, browser-engine-level isolation. Significant architectural change but future-proofs security.

13. Semantic Search in Index. *Source: A-RAG* [27]. Add embedding-based retrieval alongside keyword/symbol search. Expose as `index_search(query, mode="keyword"|"semantic"|"hybrid")`. Enables finding conceptually related code, not just exact matches.

14. Knowledge Atom Extraction (GCC). *Source: PlugMem* [11]. Extract atomic facts from commits (“MCP server has 17 tools”, “tests: 600 passing”) into a separate lightweight index. `gcc_context` returns atoms instead of full commit text for broad queries, dramatically reducing tokens.

7 Discussion

7.1 Architectural Validation

The survey provides strong external validation for CCR’s core design choices:

1. **Git-style memory is converging as standard.** Letta/MemGPT independently introduced “Context Repositories” (git-based versioning) in February 2026, and GCC v2 reports state-of-the-art results on SWE-Bench Verified.
2. **Zero-LLM-call architecture is viable.** CER [26] demonstrates training-free self-improvement, and the Codified Context paper [28] validates hot/cold memory patterns across 283 development sessions without requiring LLM calls for memory management.
3. **REPL-as-environment is validated.** The RLM REPL-only ablation [3] confirms that even without recursive sub-model calls, the REPL paradigm delivers substantial gains.
4. **Self-evolving strategies are unique.** No competing system—commercial, open-source, or academic—implements self-evolving playbooks with helpful/harmful counters. ACE remains CCR’s strongest differentiator.

7.2 Key Risks

Claude Code native memory evolution. If Anthropic adds versioning, search, or strategy evolution to Auto Memory, CCR’s advantage narrows. Mitigation: stay structurally superior (branches, merges, playbooks, REPL) rather than competing on basic persistence.

MCP marketplace fragmentation. Users may prefer assembling 3–4 focused tools rather than one comprehensive server. Mitigation: emphasize that GCC + ACE + RLM integration is the value—they work better together than as separate tools.

Embedding dependency for semantic search. Adding semantic search (Tier 3, item 13) would require either a local embedding model or an API key, potentially breaking the zero-infrastructure principle. Mitigation: use lightweight local models (e.g., all-MiniLM-L6-v2 via ONNX) or defer to keyword search when embeddings are unavailable.

7.3 Limitations of This Survey

Our analysis is limited to publicly available papers, documentation, and open-source code as of March 10, 2026. Commercial systems (Devin, Cursor) may have unpublished capabilities. Benchmark comparisons across papers are not directly comparable due to differing evaluation protocols. The proposed roadmap has not been empirically validated; impact estimates are extrapolated from source papers’ results on different tasks.

8 Conclusion

We surveyed 50+ papers, frameworks, and production systems across three domains—persistent agent memory, self-evolving strategies, and sandboxed execution—to inform the next version of CCR. Our analysis reveals that:

1. Graph-based memory with admission control (MAGMA, A-MAC) outperforms flat stores by up to 45.5% on reasoning benchmarks, motivating commit linking and novelty gating for GCC.
2. Hierarchical skill libraries with structured failure distillation (SkillRL) achieve 10–20× compression while making failures actionable, directly informing ACE’s playbook evolution.
3. Temporal decay (SYNAPSE, ACT-R) and multi-level summarization (TiMem, 52% token reduction) provide principled mechanisms for managing growing memory.
4. Kernel-level sandboxing (nono/Seatbelt) provides zero-overhead isolation categorically stronger than CCR’s current application-level blocklist.
5. CCR’s unified architecture remains unique—no competing system combines versioned memory, self-evolving playbooks, and sandboxed REPL in a single zero-LLM-call MCP server.

We propose a 14-item roadmap organized into three tiers. Tier 1 (6 items) offers the highest return on investment and can be implemented without architectural changes. Tier 2 (4 items) requires moderate refactoring but delivers significant capability improvements. Tier 3 (4 items) represents transformative changes that would position CCR at the frontier of the agent memory research agenda.

The convergence of independent research toward git-style memory (Letta), self-evolving strategies (MCE, SkillRL), and REPL-based reasoning (RLM-Qwen3) validates CCR’s foundational bet that coding agents need structured, versioned, evolving memory—not just flat text files.

References

- [1] Git Context Controller (GCC). *arXiv:2508.00031*, 2025 (revised March 2026).

- [2] Agentic Context Engineering (ACE). *arXiv:2510.04618*, 2025.
- [3] A. Zhang, T. Kraska, O. Khattab. Recursive Language Models. *arXiv:2512.24601*, December 2025.
- [4] H. Ye, X. He, et al. MCE: Meta Context Engineering via Agentic Skill Evolution. *arXiv:2601.21557*, January 2026.
- [5] SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *arXiv:2602.08234*, February 2026.
- [6] EXIF: Exploration-First Automated Skill Discovery for Language Agents. *arXiv:2506.04287*, June 2025.
- [7] MAGMA: Multi-Graph Agentic Memory Architecture. *arXiv:2601.03236*, January 2026.
- [8] SYNAPSE: Episodic-Semantic Memory via Spreading Activation. *arXiv:2601.02744*, January 2026.
- [9] A-MEM: Agentic Memory for LLM Agents. *arXiv:2502.12110*, February 2025.
- [10] Graphiti: Temporal Knowledge Graphs for AI Agents. *arXiv:2501.13956*, January 2025.
- [11] PlugMem: Task-Agnostic Plugin Memory. *arXiv:2603.03296*, March 2026 (Microsoft Research).
- [12] A-MAC: Adaptive Memory Admission Control. *arXiv:2603.04549*, March 2026.
- [13] TiMem: Temporal-Hierarchical Memory Consolidation. *arXiv:2601.02845*, January 2026.
- [14] HiMem: Hierarchical Long-Term Memory for LLM Agents. *arXiv:2601.06377*, January 2026.
- [15] MemRL: Self-Evolving Agents via Reinforcement Learning on Episodic Memory. *arXiv:2601.03192*, January 2026.
- [16] Continuum Memory Architecture. *arXiv:2601.09913*, January 2026.
- [17] Aeon: Neuro-Symbolic Memory OS for LLM Agents. *arXiv:2601.15311*, January 2026.
- [18] Memory in the Age of AI Agents: A Survey. *arXiv:2512.13564*, December 2025.
- [19] Graph-based Agent Memory: Taxonomy, Techniques, and Applications. *arXiv:2602.05665*, February 2026.
- [20] ICLR 2026 MemAgents Workshop: Memory for LLM-Based Agentic Systems. Vienna, May 2026.
- [21] MemoryAgentBench. *arXiv:2507.05257*, ICLR 2026.
- [22] MemBench: Evaluating Agent Memory. ACL 2025 Findings.
- [23] LoCoMo: Long-term Conversational Memory Benchmark. Snap Research.
- [24] AMA-Bench: Evaluating Long-Horizon Agentic Memory. *arXiv:2602.22769*, February 2026.
- [25] ACON: Optimizing Context Compression for Long-Horizon Agents. *arXiv:2510.00615*, ICLR 2025.
- [26] CER: Contextual Experience Replay for Self-Improvement. *arXiv:2506.06698*, ACL 2025.
- [27] A-RAG: Scaling Agentic RAG via Hierarchical Retrieval Interfaces. *arXiv:2602.03442*, February 2026.

- [28] Codified Context: Infrastructure for AI Agents in Complex Codebases. *arXiv:2602.20478*, February 2026.
- [29] AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. *arXiv:2506.13131*, DeepMind, May 2025.
- [30] Effective Context Engineering for AI Agents. *Anthropic Engineering Blog*, September 2025.
- [31] Code Execution with MCP. *Anthropic Engineering Blog*, November 2025.
- [32] RLM-Qwen3-8B: A Natively Recursive Language Model. 2026.
- [33] Prime Intellect. RLM as the Paradigm of 2026. *Blog post*, 2026.
- [34] OpenHands: An Open Platform for AI Software Developers. *arXiv:2407.16741*, 2024.
- [35] OpenHands Context Condensation for More Efficient AI Agents. *OpenHands Blog*, 2025.
- [36] Cognition AI. Devin: AI Software Engineer. 2024–2026.
- [37] CaMeL: Defeating Prompt Injections by Design. *arXiv:2503.18813*, Google DeepMind, 2025.
- [38] Microsoft. Wassetto: WebAssembly-based Tools for AI Agents. August 2025.
- [39] nono: Kernel-level Agent Sandboxing. *GitHub/HuggingFace*, 2026.
- [40] OWASP Top 10 for Agentic Applications. 2026 Edition.
- [41] Cloudflare. Code Mode MCP: Give Agents an Entire API in 1,000 Tokens. 2026.
- [42] Model Context Protocol Specification. Version 2025-11-25.
- [43] J. R. Anderson. How Can the Human Mind Occur in the Physical Universe? Oxford University Press, 2007.
- [44] Human-Like Remembering and Forgetting for LLM Agents. ACM HAI 2025.
- [45] From Experience to Strategy: Trainable Graph Memory for Agents. *arXiv:2511.07800*, ICLR 2026 under review.
- [46] AI Agentic Programming: A Survey. *arXiv:2508.11126*, 2025.